

1.6.1 if

The if statement evaluates an expression. If that expression is true, then a statement is executed. If an else clause is given and if the expression is false, then the else's statement is executed.

Syntax:

```
if ( expression ) statement1 ;
```

or

```
if ( expression ) statement1 ;  
else statement2 ;
```

Examples:

```
if (loop<3) counter++;
```

```
if (x==y)  
    x++;  
else  
    y++;
```

```
if (z>x)  
{  
    z=5;  
    x=3;  
}  
else  
{  
    z=3;  
    x=5;  
}
```

1.6.2 switch

A switch statement allows a single variable to be compared with several possible constants. If the variable matches one of the constants, then a execution jump is made to that point. A constant can not appear more than once, and there can only be one default expression.

Syntax:

```
switch ( variable )  
{  
    case const:  
        statements...;  
    default:  
        statements...;  
}
```

Examples:

```
switch(betty)
{
    case 1:
        printf("betty=1\n");
    case 2:
        printf("betty=2\n");
        break;
    case 3:
        printf("betty=3\n");
        break;
    default:
        printf("Not sure.\n");
}
```

If *betty* is 1, then two lines are printed: *betty=1* and *betty=2*. If *betty* is 2, then only one line is printed: *betty=2*. If *betty*=3, then only one line is printed: *betty=3*. If *betty* does not equal 1, 2, or 3, then "Not sure." is printed.

1.6.3 while

The while statement provides an iterative loop.

Syntax:

```
while( expression ) statement...
```

statement is executed repeatedly as long as *expression* is true. The test on *expression* takes place before each execution of *statement*.

Examples:

```
while(*pointer!='j') pointer++;

while(counter<5)
{
    printf("counter=%i", counter);
    counter++;
}
```

1.6.4 do

The do...while construct provides an iterative loop.

Syntax:

```
do statement... while( expression );
```

statement is executed repeatedly as long as *expression* is true. The test on *expression* takes place after each execution of *statement*.

Examples:

```
do {
    betty++;
    printf("%i",betty);
} while (betty<100);
```

1.6.5 for

The for statement allows for a controlled loop.

Syntax:

for (expression1 ; expression2 ; expression3) statement...

expression1 is evaluated before the first iteration. After each iteration, *expression3* is evaluated. Both *expression1* and *expression3* may be omitted. If *expression2* is omitted, it is assumed to be 1. *statement* is executed repeatedly until the value of *expression2* is 0. The test on *expression2* occurs before each execution of *statement*.

Examples:

```
for(loop=0;loop<1000;loop++)
    printf("%i\n",loop);
```

Prints numbers 0 through 999.

```
for(x=3, y=5; x<100+y; x++, y--)
{
    printf("%i\n",x);
    some_function();
}
```

Prints numbers 3 through 53. `some_function` is called 51 times.

1.6.6 goto

The goto statement transfers program execution to some label within the program.

Syntax:

```
goto label;
....
label:
```

Examples:

```
goto skip_point;
```

```
printf("This part was skipped.\n");  
skip_point:  
printf("Hi there!\n");
```

Only the text "Hi there!" is printed.

1.6.7 continue

The continue statement can only appear in a loop body. It causes the rest of the statement body in the loop to be skipped.

Syntax:

```
continue;
```

Examples:

```
for(loop=0;loop<100;loop++)  
{  
    if(loop==50)  
        continue;  
    printf("%i\n",loop);  
}
```

The numbers 0 through 99 are printed except for 50.

```
joe=0;  
while(joe<1000)  
{  
    for(zip=0;zip<100;zip++)  
    {  
        if(joe==500)  
            continue;  
        printf("%i\n",joe);  
    }  
    joe++;  
}
```

Each number from 0 to 999 is printed 100 times except for the number 500 which is not printed at all.

1.6.8 break

The break statement can only appear in a switch body or a loop body. It causes the execution of the current enclosing switch or loop body to terminate.

Syntax:

```
break;
```

Examples:

```
switch(henry)
{
    case 1: print("Hi!\n");
            break;
    case 2: break;
}
```

If henry is equal to 2, nothing happens.

```
for(loop=0;loop<50;loop++)
{
    if(loop==10)
        break;
    printf("%i\n",loop);
}
```

Only numbers 0 through 9 are printed.

1.6.9 return

The return statement causes the current function to terminate. It can return a value to the calling function. A return statement can not appear in a function whose return type is void. If the value returned has a type different from that of the function's return type, then the value is converted. Using the return statement without an expression creates an undefined result. Reaching the } at the end of the function is the same as returning without an expression.

Syntax:

```
return expression;
```

Examples:

```
int alice(int x, int y)
{
    if(x<y)
        return(1);
    else
        return(0);
}
```